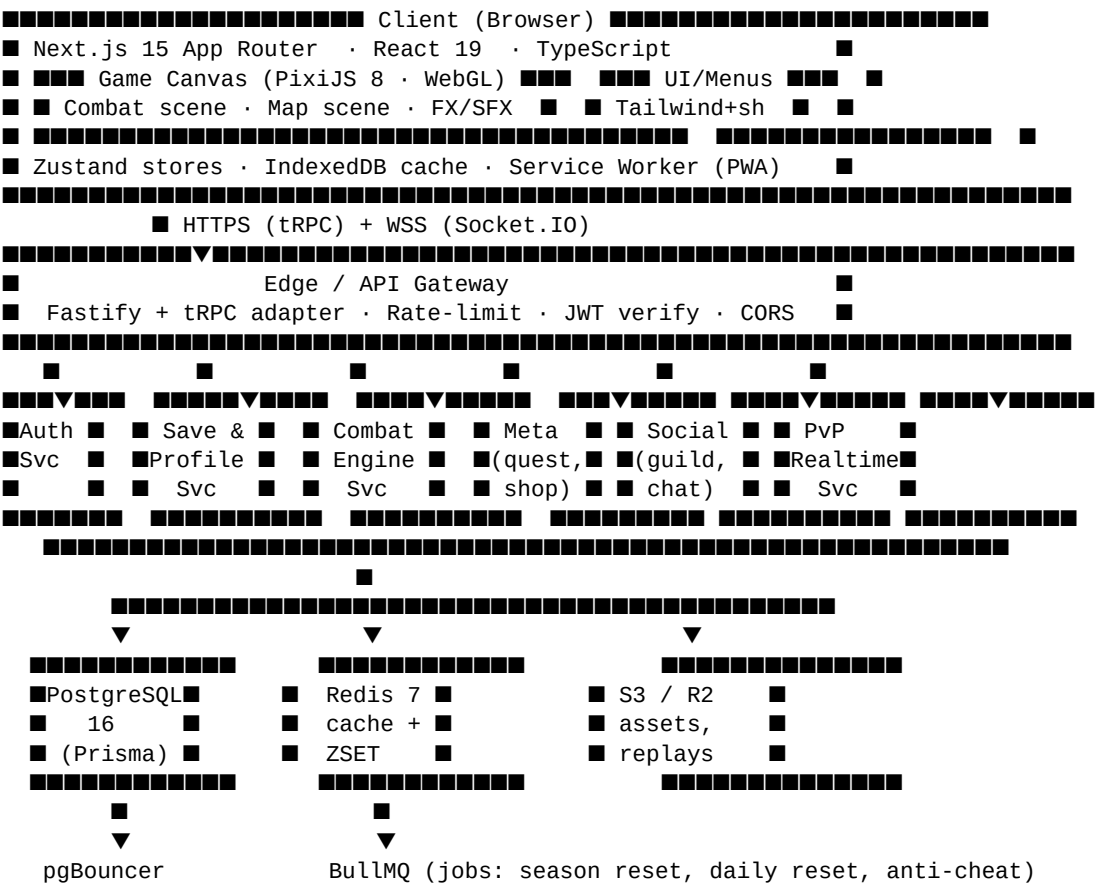


AETHERIA — Architecture Design

Step 2 deliverable · derives from 01_GAME_GUIDELINE.md

1. Architecture Style

Modular monolith first → microservices when traffic justifies it. Why: rapid iteration, single deploy, type safety end-to-end. Boundaries inside the monolith are enforced by package isolation (Nx/Turborepo) and an internal event bus, so we can later peel a service off (e.g. `pvp-realtime`) with minimal pain.



2. Layers

Layer	Responsibility
Presentation	UI, rendering, local state, optimistic updates, animations
API Gateway	Auth check, rate limit, schema validation, tracing
Application	Use-cases (orchestrate domain ops, cross-domain transactions)
Domain	Pure business rules: combat, level, quest, ranking (no I/O)
Infrastructure	DB repositories, Redis, file storage, message bus

3. Module Boundaries (monorepo packages)

apps/

```

web/          # Next.js client
api/          # Fastify + tRPC API
realtime/     # Socket.IO server (PvP, chat)
worker/       # BullMQ workers (cron, anti-cheat, season reset)
packages/
  domain-combat/    # Pure TS combat engine (also runs in client for prediction)
  domain-progression/ # XP, level, ascension rules
  domain-economy/   # Items, marketplace, currency
  domain-social/   # Guild, friends, chat moderation
  schema-db/        # Prisma schema + migrations
  schema-api/       # tRPC routers, Zod validators
  ui-kit/           # Shared React components, theme tokens
  game-assets/      # Manifest of asset URLs, level definitions
  shared-types/     # cross-cutting TS types
  config/           # eslint, tsconfig, tailwind preset

```

4. Realtime Architecture

- **WebSocket** via Socket.IO (room per match / guild / chat channel).
- PvP: server is authoritative; client predicts then reconciles on diff.
- Heartbeats every 5 s; if missed >15 s, opponent wins by forfeit.
- Sticky sessions on the realtime tier (Redis adapter for horizontal scale).

5. Save / Cloud-Sync Strategy

- **Two-tier**:
- **Local IndexedDB** snapshot after every state change (instant resume offline).
- **Server delta sync** every 30 s or on idle / page-hide.
- **Conflict policy**: server timestamp wins; client merges cosmetic (settings, photo album) by union.
- **Slots**: slot=0 is autosave; slots 1–3 are manual.
- **Schema versioning** in payload; migration runs on read.

6. Anti-Cheat & Integrity

- All combat actions validated server-side using the same `domain-combat` package.
- Replay log of every action stored in `runs.action_log`. Worker scans for impossible states.
- Rate limits per IP + per user.
- Item/currency mutations only via signed events from server logic; client never asserts deltas.
- HMAC-signed asset manifest to detect modded clients.

7. Observability

- **Tracing**: OpenTelemetry → Tempo / Jaeger.
- **Metrics**: Prometheus (latency, queue depth, MMR distribution).
- **Errors**: Sentry (JS + Node).
- **Audit**: every privileged op (currency change, ban, role change) → `audit_log`.

8. Security Baseline

- HTTPS-only, HSTS, CSP, SameSite=Lax cookies, refresh-token rotation.
- Argon2id password hashing; OAuth via NextAuth.
- Zod validation at every API boundary.
- SQL: parameterized via Prisma; no raw concatenation.
- Secrets via env + Doppler/Vault; never committed.
- OWASP Top 10 baseline; quarterly dependency audit (`pnpm audit`, Snyk).

9. Deployment Topology

Tier	Recommended host	Notes
Frontend	Vercel (or Cloudflare Pages)	Edge SSR, automatic CDN
API	Fly.io / Render	Multi-region, autoscale
Realtime	Fly.io	Sticky session, Redis adapter
Worker	Fly.io / Railway	Cron + queue
Postgres	Neon (serverless) or RDS	pgBouncer, daily backup, 7-day PITR
Redis	Upstash / ElastiCache	TTL keys for sessions; ZSET leaderboards
Object store	Cloudflare R2	Asset CDN

10. Scaling Plan

- 0 → 10 k DAU: single region, monolith.
- 10 k → 100 k: split realtime, add read-replica, ZSET per region.
- 100 k+: shard PvP by region, queue-based ranking calc, ClickHouse for analytics.